

# **B.Tech. BCSE497J - Project-I**

## **PneuNet: PNEUMONIA DETECTION USING CNN**

*Submitted in partial fulfillment of the requirements for the degree of*

### **Bachelor of Technology**

*in*

### **Programme**

*by*

**22BDS0269**

**SWAPNIL DWIVEDI**

**22BDS0350**

**ADARSH TIWARI**

**Under the Supervision of**

**Prof. Sudhakar P**

Assistant Professor Sr. Grade 1

School of Computer Science and Engineering (SCOPE)



**VIT<sup>®</sup>**  
**Vellore Institute of Technology**  
(Deemed to be University under section 3 of UGC Act, 1956)

November 2025

## **DECLARATION**

I hereby declare that the project entitled PneuNet: PNEUMONIA DETECTION USING CNN submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. / Dr. Sudhakar P.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 5/11/25

**Signature of the Candidate**

## **CERTIFICATE**

This is to certify that the project entitled PneuNet: PNEUMONIA DETECTION USING CNN submitted by Swapnil Dwivedi (22BDS0269), Adarsh Tiwari (22BDS0350), **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by him / her under my supervision during Fall Semester 2024-2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :5/11/2025

**Signature of the Guide**

**Examiner(s)**

**DR. SENDHIL KUMAR K S**

**B. TECH**

## **ACKNOWLEDGEMENTS**

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to Sendhil Kumar K S, the Head of the Data Science, for his/her insightful guidance and continuous support. His expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. His constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project guide, Prof. Sudhakar P, for his dedicated mentorship and invaluable feedback. His patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share his/her expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. His/her support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

**Swapnil Dwivedi  
Adarsh Tiwari**

## TABLE OF CONTENTS

| SL.NO     | CONTENTS                             | PAGE NO.  |
|-----------|--------------------------------------|-----------|
|           | <b>ABSTRACT</b>                      | <b>7</b>  |
| <b>1.</b> | <b>INTRODUCTION</b>                  | <b>8</b>  |
|           | 1.1 BACKGROUND                       | 8         |
|           | 1.2 MOTIVATIONS                      | 8         |
|           | 1.3 SCOPE OF THE PROJECT             | 9         |
| <b>2.</b> | <b>PROJECT DESCRIPTION AND GOALS</b> | <b>10</b> |
|           | 2.1 LITERATURE REVIEW                | 10        |
|           | 2.2 RESEARCH GAP                     | 11        |
|           | 2.3 OBJECTIVES                       | 12        |
|           | 2.4 PROBLEM STATEMENT                | 12        |
|           | 2.5 PROJECT PLAN                     | 13        |
| <b>3.</b> | <b>TECHNICAL SPECIFICATION</b>       | <b>15</b> |
|           | 3.1 REQUIREMENTS                     | 15        |
|           | 3.1.1 FUNCTIONAL                     | 15        |
|           | 3.1.2 NON-FUNCTIONAL                 | 16        |
|           | 3.2 FEASIBILITY STUDY                | 16        |
|           | 3.2.1 TECHNICAL FEASIBILITY          | 16        |
|           | 3.2.2 ECONOMIC FEASIBILITY           | 17        |
|           | 3.2.2 SOCIAL FEASIBILITY             | 18        |
|           | 3.3 SYSTEM SPECIFICATION             | 18        |
|           | 3.3.1 HARDWARE SPECIFICATION         | 18        |
|           | 3.3.2 SOFTWARE SPECIFICATION         | 19        |
| <b>4.</b> | <b>DESIGN APPROACH AND DETAILS</b>   | <b>20</b> |
|           | 4.1 SYSTEM ARCHITECTURE              | 20        |
|           | 4.2 DESIGN                           | 20        |
|           | 4.2.1 DATA FLOW DIAGRAM              | 20        |
|           | 4.2.2 USE CASE DIAGRAM               | 21        |
|           | 4.2.3 CLASS DIAGRAM                  | 21        |
|           | 4.2.4 SEQUENCE DIAGRAM               | 22        |
| <b>5.</b> | <b>METHODOLOGY AND TESTING</b>       | <b>24</b> |

|     |                                                     |    |
|-----|-----------------------------------------------------|----|
| 5.1 | MODULE DESCRIPTION                                  | 24 |
| 5.2 | TESTING                                             | 24 |
| 6.  | PROJECT DEMONSTRATION                               | 25 |
| 7.  | RESULT AND DISCUSSION (COST ANALYSIS AS APPLICABLE) | 27 |
| 8.  | CONCLUSION                                          | 33 |
| 9.  | REFERENCES                                          | 35 |
|     | APPENDIX A – SAMPLE CODE                            | 37 |

## **ABSTRACT**

Pneumonia is an infection of the lungs that causes a major health problem globally and is still the main reason for death among children under five years old and in the elderly. People with the disease have an inflammation of their lungs accompanied by difficulty in breathing due to liquid accumulation. The problem may even result in a fatal outcome. Detecting pneumonia early is therefore crucial in order to administer treatment early enough and to have better chances of recovery. But a diagnosis using a chest X-ray is not done directly as it needs the expertise of a skilled radiologist. It is relatively slow, labor-intensive, and susceptible to errors of interpretation.

This work is about the construction of a system that can automatically diagnose pneumonia by using deep learning methods, mainly the CNN model. The CNN learns the difference between two kinds of images, i.e., chest X-ray images of Normal and Pneumonia patients, and then classifies the images into one of the two groups. In doing so, the CNN analyses visual features of the healthy lungs and the ones that are infected in the X-ray images. Along with these, the image pixels are also scaled down in value, the images are uniformly sized and cleaned of random noise in order to facilitate the model to learn features well. Further, the authors of the paper resort to different techniques such as rotating, flip, and scaling to diversify the dataset and improve the model's power for untested images from the test sets.

The CNN model constructed here incorporates several strategies such as batch normalization, dropout, and early stopping to lessen overfitting and increase the model's precision. The model, after training and testing, is highly capable of identifying situations where the lungs are infected with pneumonia resulting from either bacteria or viruses. The performance of the system, gauged using metrics, such as accuracy, and sensitivity, is solid and thus, the method can be considered as one of the aids in diagnostics.

The technique that is put forward here achieves automated image-based diagnosis and hence the turnaround time is reduced considerably thus; the solution is more affordable, and healthcare centers in underdeveloped areas without medical experts will benefit a lot from it. The task load of the radiologists is reduced, and the provision of some preliminary information on the screening facilitates clinical decision-making. To sum up, this research demonstrates the potential of AI and computer vision to revolutionize medical imaging and thus be in line with other CAD systems, contributing to a faster diagnosis of respiratory diseases and enhancing healthcare quality.

## **Keywords**

Pneumonia Detection, Chest X-ray, Convolutional Neural Networks, Deep Learning, Image Classification, Medical Imaging, Data Augmentation, Batch Normalization, Early Stopping, Computer-Aided Diagnosis, Healthcare AI.

# CHAPTER 1

## INTRODUCTION

### 1.1 Background

Pneumonia is a serious inflammatory condition of the lungs caused by bacteria, viruses, or fungi, that leads to the lungs' alveoli filling with fluid. Although chest X-rays are the main tool used to diagnose pneumonia, calling them correctly needs highly skilled radiologists and can be a very long process, which can also be affected by errors of human nature.

Driven by the advances in deep learning and artificial intelligence, the automated image analysis is considered a viable solution for the medical diagnostics. Convolutional Neural Networks (CNNs) have shown an impressive ability to identify intricate patterns in medical images, thus, making disease detection both faster and more accurate.

This custom CNN model is created in the project and is merged with the pretrained model architectures to get the advantages of both transfer learning and domain-specific learning. While the pretrained models offer robust feature extraction capabilities, the custom CNN layers adjust the network to the specific features of pneumonia in a chest X-ray. Such a hybrid strategy allows the network to be more robust and accurate, thereby releasing an efficient and consistent system for detecting pneumonia.

### 1.2 Motivations

Pneumonia is still among the major factors leading to death all over the globe, regularly being the main cause of death for children aged under five and the elderly. In line with the data from the World Health Organization (WHO), there are newly confirmed cases in the millions every year, and a large number of deaths result from the late and/or inaccurate diagnosis. Though the chest X-ray is considered a primary examination method, visualizing these photos heavily relies on the expertise of the radiologist and the results may differ under various circumstances such as tiredness, amount of work, and lack of facilities.

This initiative's reasons are three major points:

- **Bridging the Healthcare Gap:** In places where medical resources are limited, the lack of skilled radiologists is typical. A computerized pneumonia identification module can be the medical staff's aid in decision-making, thus the diagnosis becomes accessible and fast enough to patients even if they are far from the hospital.
- **Reducing Diagnostic Errors:** The X-ray manual reading can be influenced by the subjectivity of the person and errors may occur. In this way deep learning instruments especially CNNs can

be used to make the system consistent and at the same time reduce the irregularities in the system, hence patient outcomes will be better.

- **Real-Time and Scalable Solutions:** Despite the accuracy, the conventional diagnostic approach takes a long time to perform and it's quite difficult to scale. The CNN-supported identification tool can deliver near to instant and precise results, thereby becoming feasible to implement in medical institutions, and m-health applications, as well as in the telemedicine sector for on-the-fly usage.

The project, through this concern, looks forward to producing a smart, cost-effective and scalable solution that not only helps the work of the radiologists, but also makes a call to the healthcare infrastructure enabling it to save more lives because of early detection and the proper treatment of the diseases.

### **1.3 Scope of the Project**

This project is limited to the binary classification of chest X-ray images, to be categorized as:

- Normal (lungs without disease)
- Pneumonia (lungs with infection)

Project pipeline:

- Preprocessing of chest X-ray images (resizing, cleaning, augmentation).
- Creation and training of a CNN model with batch normalization and dropout.
- Using early stopping to prevent overfitting.
- Performance evaluation of the model on train, validation, and test datasets.
- Visualizing training performance and dataset distribution.

## CHAPTER 2

### PROJECT DESCRIPTION AND GOALS

#### 2.1 Literature Review

##### 1. Introduction to Pneumonia Detection

- Pneumonia is still one of the major causes of illness and death globally. The conventional ways to identify the disease, e.g., the manual reading of chest X-ray (CXR) images, take a lot of time and the results may vary depending on the person. To overcome these problems, the use of deep learning (DL) techniques has been explored recently. DL methods can be used for fast and reliable automated detection of pneumonia from CXR images.

##### 2. Deep Learning Architectures for Pneumonia Detection

- **Convolutional Neural Networks (CNNs):** CNNs are the go-to method for most image classification tasks. R. Kailasam et al. (2025) in their work used CNNs for the automated detection and localization of pneumonia, which in turn they verified was a rapid process with high diagnostic accuracy and precision in a real clinical setting.
- **DenseNet-121 with CBAM:** An advanced model which fused the Convolutional Block Attention Module (CBAM) with DenseNet-121 was put forward by AB Rashid et al. (2025). Their work had the effect of thoroughly extracting features by employing both channel and spatial attention thereby test accuracy reached 98.64%.
- **Transfer Learning Models:** The use of pre-trained models such as ResNet50, InceptionV3, and InceptionResNetV2 for pneumonia detection is one of the promising directions. These models employ transfer learning which results in better accuracy and less training time.

##### 3. Challenges in Pneumonia Detection

- **Imbalanced Datasets:** Differences between the number of normal and pneumonia cases in datasets may cause models to be biased towards the more frequent class. F. Alshanketi (2024) pointed out that fixing this imbalance was the key factor in improving model performance.
- **Interpretability:** Understanding model decisions is crucial in medical applications. Grad-CAM++ has been utilized to visualize regions of interest in CXR images, aiding in model interpretability.

##### 4. Evaluation Metrics

- **Accuracy:** Degree of correctness of predictions in general.
- **Precision:** Fraction of correctly predicted pneumonia cases among all predicted positive cases.
- **Recall:** Fraction of correctly identified pneumonia cases among actual positive samples.
- **F1-Score:** Harmonic mean of precision and recall.

- **Confusion Matrix:** Visual breakdown of correct and incorrect classifications.

## 5. Future Directions

- **Real-Time Systems:** Building systems that can perform real-time pneumonia detection in a hospital environment.
- **Explainable AI:** Improving the clarity of the model to get the trust of the medical staff.

## 2.2 Research Gap

### Limited Generalization of Models to Different Datasets

- The majority of CNN-based models (CheXNet, DenseNet, ResNet), have been developed by using data from single medical centers, such as Kaggle Pediatric CXR or ChestX-ray14.
- As a result, their accuracy is usually lowered when they are evaluated on data that come from different sources since the equipment used for imaging, the characteristics of patients, and the conditions of the acquisition may differ.
- **Gap:** There is a demand for models that can perform robustly across various hospitals and X-ray machines.

### Insufficient Explainability

- Grad-CAM and Grad-CAM++ methods can show some visual interpretability of the working of the model, but in general, the majority of the models are "black boxes".
- Doctors require explicit explanations of model predictions to be able to rely on AI-assisted diagnosis.
- **Gap:** Creating significantly more transparent models that can not only highlight the areas but do so in a way that is clinically relevant and reliable.

### Data Imbalance and Limited Dataset Size

- Several studies emphasize the problem of class imbalance (the number of pneumonia cases is less than the number of normal cases) as well as the limitation of annotated datasets.
- Such a situation may result in biased predictions, overfitting, and low sensitivity of the minority classes.
- **Gap:** There is a requirement for strong augmentation techniques, generating synthetic data, or datasets from multiple institutions.

### Real-Time Deployment Challenges

- The majority of current models have been trained and tested using high-end hardware. However, there is very little work done regarding their deployment on edge devices or in real-time clinical environments.
- **Gap:** Create lightweight, power-saving models that can perform real-time pneumonia detection in hospital settings or on point-of-care devices.

### Multi-Disease Detection Capability

- The majority of the research works have a strong focus on binary classification, i.e., Differentiating between Normal and Pneumonia.
- From a clinical perspective, it is of utmost importance to be able to tell the difference between pneumonia and other lung diseases such as tuberculosis, COVID-19, and

ARDS.

- **Gap:** Develop models that can recognize multiple classes which will provide broader diagnostic capabilities.

### **Limited Localization and Clinical Validation**

- Some research works (for example, Kailasam et al., 2025) that utilized YOLO for ROI localization, while most classification models only infer image-level predictions.
- Clinical validation with radiologists and their involvement in the workflow is still very limited.
- **Gap:** Merging detection, localization, and explainability, along with thorough clinical trials.

## **2.3 Objectives**

The main purpose of this work is to create and execute an automated deep learning system that can recognize pneumonia in chest X-ray images through an ensemble model of a custom CNN and a ResNet152V2 pretrained network.

The specific objectives are:

- To create a custom CNN architecture that can understand spatial and texture-based features of the lung areas infected with pneumonia.
- To employ the ResNet152V2 pretrained model for transfer learning, benefiting from its deep feature extraction potential to augment model performance and lessen training complexity.
- To prepare and augment the dataset through normalization, resizing, and data augmentation methods to generalize better and prevent overfitting.
- To adjust the combined architecture to optimal hyperparameters that allow a trade-off between computational efficiency and accuracy.
- To measure model performance through different standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix.
- To design a powerful and trustworthy deep learning system able to facilitate the work of radiologists and support them in making fast and accurate diagnoses of pneumonia from X-ray images.

## **2.4 Problem Statement**

Pneumonia ranks as one of the major causes of death globally, severely impacting mostly children and the elderly. The conventional way to diagnose the disease depends on the analysis of chest X-ray which is done by radiologists. However, this process takes a lot of time, is subject to different opinions, and may contain mistakes. Besides, a shortage of skilled radiologists is reported in many remote and less developed areas, thus the detection and treatment in time become a challenge.

A deep learning method, especially a Convolutional Neural Network (CNN), has noticeably raised a medical image classification level, but still, the need for a local, trustworthy, and easy-to-use system, capable of recognizing pneumonia automatically in chest X-ray images, remains open. The main difficulties are the handling of various image qualities, achieving a high level of accuracy in classification, and providing an easily understandable output to be able to help

the medical professionals in their clinical decision-making.

As such, an automated pneumonia detection system using image preprocessing, CNN-based classification, and real-time prediction is essential to be integrated with the medical care professionals to ease their work, lower the diagnosis time, and elevate the patients' well-being.

## 2.5 Project Plan

The project phases were designed to ensure a structured and efficient workflow that would take the team through the project from the initial planning stage all the way to the final documentation stage. Each phase was made up of specific tasks that had their timelines and deliverables specified.

### 2.5.1 Project Initiation & Planning

The focus of this preparatory phase was on the definition of the project's scope, objectives, and overall strategy. This was very important in ensuring that the whole team was on the same page and that all the project requirements were understood clearly before the development phase started.

- **Project Definition:** The main problem statement, project goals, and high-level objectives were set up.
- **Literature Review & Gap Analysis:** The team conducted an exhaustive review of the state-of-the-art research in pneumonia detection through chest X-rays to understand the extent of the existing research, recognize the research gaps, and find out how this project would contribute to the field.
- **SRS (Software Requirements Specification):** An elaborate SRS document was put together that described all the functional and non-functional requirements of the system, including the expected performance and accuracy metrics.

### 2.5.2 Data & Model Development

This was the core technical phase, where the deep learning pipeline was developed and implemented.

- **Dataset Acquisition:** The necessary chest X-ray dataset was obtained and checked for quality and diversity.
- **Data Preprocessing:** The unprocessed data was normalized and resized to a standard format. The training dataset was expanded by applying data augmentation methods, and the data was divided into training, validation, and testing sets.
- **Model Architecture:** A Custom CNN architecture was created and combined with the ResNet152V2 pretrained model. The network layers, activation functions, and loss functions were defined to guide training effectively.
- **Model Training:** The hybrid model was trained on the processed dataset for a specified number of epochs, with validation performance being constantly checked to prevent overfitting and ensure stable convergence.

### 2.5.3 Evaluation & Analysis

During this phase, the model that had been trained was put through a very demanding test and the results it produced were carefully analyzed.

- **Model Evaluation:** The trained models were scored on the test dataset with metrics like accuracy, precision, recall, F1-score, and confusion matrix. The training and validation curves were inspected to evaluate generalization and to spot any overfitting or underfitting.
- **Results Analysis:** The performance metrics were used to uncover the model's strengths, weaknesses, and the instances of misclassifications at the level of classes. A close look was taken at the model's capability to differentiate between normal and pneumonia cases.
- **Reliability Assessment:** The model's agreement and robustness with different test samples were confirmed to be reliable classification results.

### 2.5.4 Finalization & Documentation

The final phase concentrated on gathering and polishing all the project deliverables into a full and professional package.

- **Final Project Report:** This comprehensive report documented all findings, methodologies, and results from the previous phases.
- **Project Presentation:** To visually present the project's goals, methods, and main findings, a slide deck was prepared.
- **Codebase Cleanup:** The codebase of the entire project was looked through, structured, and commented for the ease of understanding, continuation, and duplication of the work.
- **Final Submission:** The team accomplished the submission of all the deliverables that comprised the source code, final report, and presentation materials, for the evaluation.

## CHAPTER 3

### TECHNICAL SPECIFICATION

### 3. REQUIREMENT

#### 3.0 User Requirements (UR)

The system was engineered to satisfy the following user demands:

- **UR-1:** Implement a swift and automated method of detection of pneumonia based on the chest X-ray images, employing deep learning.
- **UR-2:** Make sure that the outcomes are trustworthy, repeatable, and have clinical significance.
- **UR-3:** Decrease the reliance on the manual diagnosis method, thus, clinicians will be facilitated in making quick decisions.
- **UR-4:** Allow the batch processing of numerous X-ray images to be done effectively without the need for a manual intervention.

#### 3.1.1 Functional Requirements (FR)

The system automated tasks in the following manner:

##### **FR-1: Image Preprocessing**

- The system accepted chest X-ray images in generally used formats like PNG, and JPEG, etc.
- The system normalized pixel values and resized the images to a standard size appropriate for the input of the model.
- Applied **data augmentation techniques** (flips, rotations, brightness and contrast adjustments) to improve model robustness and prevent overfitting.

##### **FR-2: Pneumonia Detection**

- As a result of the hybrid deep-learning model, a combination of a Custom CNN and ResNet152V2 pretrained network was created for precise classification.
- To get more exact and to avoid overfitting the authors of the paper have performed fine-tuning of the ResNet152V2 layers and optimized custom dense layers for feature extraction.
- Normal and Pneumonia were the two categories in which input images were classified.
- Also, for each prediction, a probability score was formed to indicate the confidence level.

##### **FR-3: Explainability & Visualization**

- The system used visualization methods to clarify the parts of the input that were most important and to provide visual feedback of the model's focus area.
- The classification results with the corresponding visualization were presented to provide more lucid understanding.

##### **FR-4: Output Generation**

- The system kept the X-rays that were either annotated or visualized to indicate the areas

infected by pneumonia.

- Made summary reports comprising predicted class, probability scores, and visualization outputs.

### **3.1.2 Non-Functional Requirements (NFR)**

These requirements were the system's quality, performance, and usability during its development and deployment:

#### **Performance:**

- Average processing time per image: < 5 seconds on a GPU in Google Colab.

#### **Accuracy:**

- Obtained classification accuracy of 80% or more on validation datasets.
- Set as the goal precision, recall, and F1-score to be balanced and trustworthy metrics of prediction.

#### **Reliability:**

- If the same inputs were tested over and over again, the results were identical.

#### **Maintainability:**

- The codebase was broken down into components, properly documented, and organized, thus it was easy to update and debug.

#### **Usability:**

- Offered an easy-to-use interface on Google Colab for image upload and classification result viewing with visualization.
- Works equally well in a local environment or in the cloud.

#### **Scalability:**

- Have a plan to be able to integrate with bigger datasets or further research multi-disease detection.

## **3.2 Feasibility Study**

The feasibility study assesses if the proposed pneumonia detection system can be developed and implemented effectively in aspects of technology, cost, and societal impact. It serves as a tool to decide the desirability of the project in the field of practice before its real implementation.

### **3.2.1 Technical Feasibility**

Technical feasibility entails the availability of tools, technologies, and technical proficiency to create the system. The proposal includes deep learning strategies like ResNet152V2 (A pretrained convolutional neural network) and Custom CNN to perform binary image classification. At the same time, the model is realized using popular frameworks such as TensorFlow and Keras that provide compatibility and scalability features. The chest X-ray images used to create the dataset are from the public domain, and they undergo preprocessing and normalization before the training phase. The modification of the pretrained model allows the system to utilize image features that are already learned, thus saving the time of training and enhancing the accuracy of the model. The facilities demanded are of the least and may be fulfilled by a system with a GPU or a Google Colab environment for quick calculations. The staging is equipped with the full gear such as data preprocessing, model training, performance evaluation, and visualization, thus confirming that the setting is technically doable.

Thus, this project is technically achievable as it employs effective algorithms, community-supported tools, and modest hardware prerequisites.

### **3.2.2 Economic Feasibility**

Economic feasibility studies the cost-benefit aspect of the system, thus ensuring that its benefits are greater than the costs related to its development and operation. The device in question would mainly involve no-cost software packages and datasets, thereby negating any software or licensing bills.

Work and trials were done on Google Colab that offers free GPU acceleration thus cutting hardware costs.

There is zero new money needed for dataset buying or for the procurement of the specialized tools.

The promise of healthcare — a fast and accurate pneumonia diagnosis — is definitely worth the low development cost.

After the model is implanted, it could be used to automate regular screening tasks in the medical field, thus saving doctors' time and cutting down on the overall diagnostic costs.

Therefore, the technology is an economic one, since it generates a large output with little or no extra costs.

### **3.2.3 Social Feasibility**

Social feasibility reviews a system's match with its target environment and the level of acceptance by users and other stakeholders.

The primary purpose of the tool is to offer healthcare personnel a quick and dependable method for confirming pneumonia.

Errors by humans are minimized to the extent that first detection is quickened and thus proper healthcare services are ensured.

With the help of technology, health workers can be more productive in tackling the cases that need their immediate attention.

By making radiology expertise in remote medical areas available through AI, the system helps achieve social good.

The outputs of the model are clear, understandable, and accompanied by graphical explanations, thereby promoting users' trust and approval.

Accordingly, the apparatus is socially acceptable as it works well with public health objectives, facilitates access, and heightens diagnostic assistance.

## **3.3 System Specification**

The system specifications outline the technical requirements that are necessary for the design, development, and implementation of a pneumonia detection system.

### **3.3.1 Hardware Specification**

#### **Component Specification**

|           |                                                       |
|-----------|-------------------------------------------------------|
| Processor | Intel Core i5 / i7 or equivalent                      |
| RAM       | Minimum 8 GB (Recommended: 16 GB for faster training) |
| Storage   | At least 20 GB of free disk space                     |
| GPU       | NVIDIA GPU (with CUDA support) or Google Colab GPU    |
| Display   | Full HD resolution for visualization outputs          |

In addition, it is possible to run all the experiments on cloud-based platforms such as Google

Colab, thus no high-end local hardware is required.

### 3.3.2 Software Specification

| Component               | Specification                                                      |
|-------------------------|--------------------------------------------------------------------|
| Operating System        | Windows 10 / 11, macOS, or Linux                                   |
| Programming Language    | Python 3.8+                                                        |
| Frameworks/Libraries    | TensorFlow, Keras, NumPy, Pandas, Matplotlib, OpenCV, scikit-learn |
| Development Environment | Google Colab / Jupyter Notebook                                    |
| Dataset Format          | PNG, JPEG (Chest X-ray Images)                                     |
| Visualization Tools     | Grad-CAM / Heatmap Visualization using OpenCV and Matplotlib       |

The chosen instruments and frameworks are also open-source, effective, and have a large community of developers, which makes them perfect for deep learning model creation.

## **CHAPTER 4**

### **Design Approach and Detail**

#### **4.1 System Architecture**

Layered Architecture for Pneumonia Detection:

Input Layer:

An image is uploaded, validated, and stored in a temporary buffer.

Processing Layer:

Preprocessing: the image is resized, normalized, and augmented.

Segmentation (U-Net): helps the model focus on the lung area by isolating it.

Feature Extraction: a custom CNN + ResNet50V2 for hierarchical and deep features.

Decision Layer:

Images are classified as Normal or Pneumonia.

Probability scores are created and saved for later analysis.

Output Layer:

X-ray images annotated with Grad-CAM.

Result reporting module with downloadable summary for medical review.

Feedback & Storage Layer:

Predictions, images, and logs are stored for the next round of model retraining and for audit purposes.

Enables the model to continuously learn and get better.

#### **4.2 Design**

##### **4.2.1 Data Flow Diagram**

DFD shows the movement of data within the pneumonia detection system. It basically describes the journey of the raw X-ray images through the different stages like preprocessing, segmentation, and finally, classification. The transitions between the modules such as data

input, preprocessing, CNN-based feature extraction, and output prediction are depicted vividly.

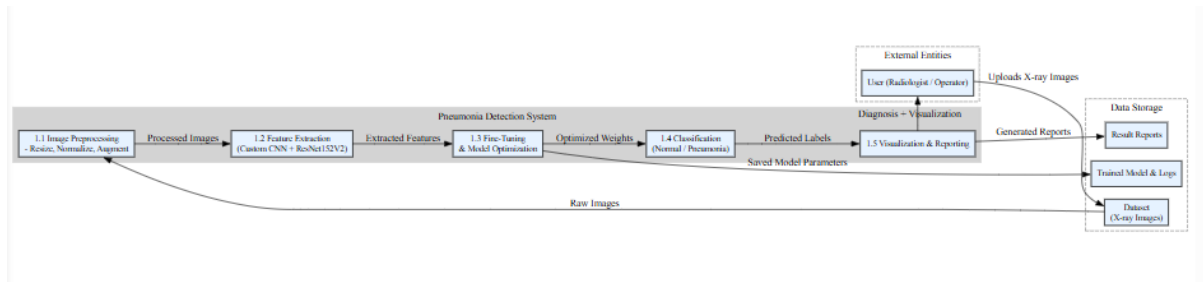


Fig. Data Flow Diagram

#### 4.2.2 Use Case Diagram

This diagram represents the interaction between the system and the external actors (e.g., doctor, radiologist, system admin). It mainly features the functionalities of X-ray uploading, processing, prediction generation, and result reviewing. It is a high-level representation of system operations.

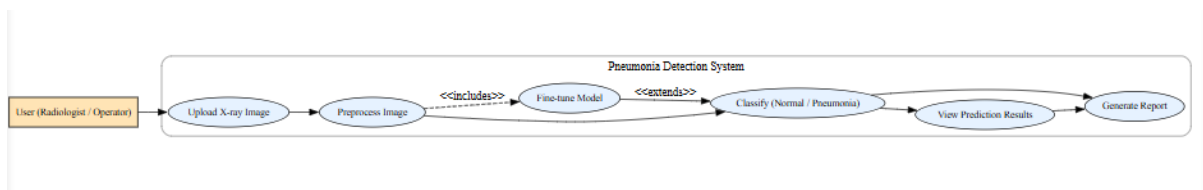


Fig. Use Case Diagram

#### 4.2.3 Class Diagram

A class diagram is the one that describes the system's static structure and the classes like ImageLoader, Preprocessor, CNNModel, ResNetModel, ResultAnalyzer. Basically, it has the class attributes and methods along with relationships, which explain the object-oriented approach.

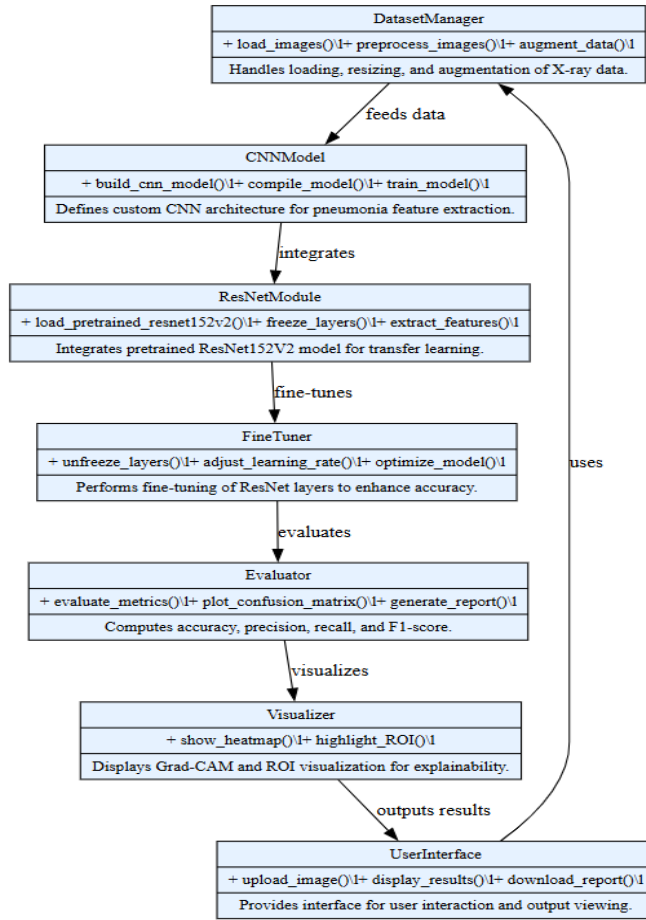


Fig. Class Flow Diagram

#### 4.2.4 Sequence Diagram

The sequence diagram explains the step-by-step process of pneumonia detection by showing the interactions between objects. These interactions include image input to preprocessing, CNN/ResNet inference, and final result output. So, the diagram is quite helpful in knowing the system's runtime behavior and the communication flow.

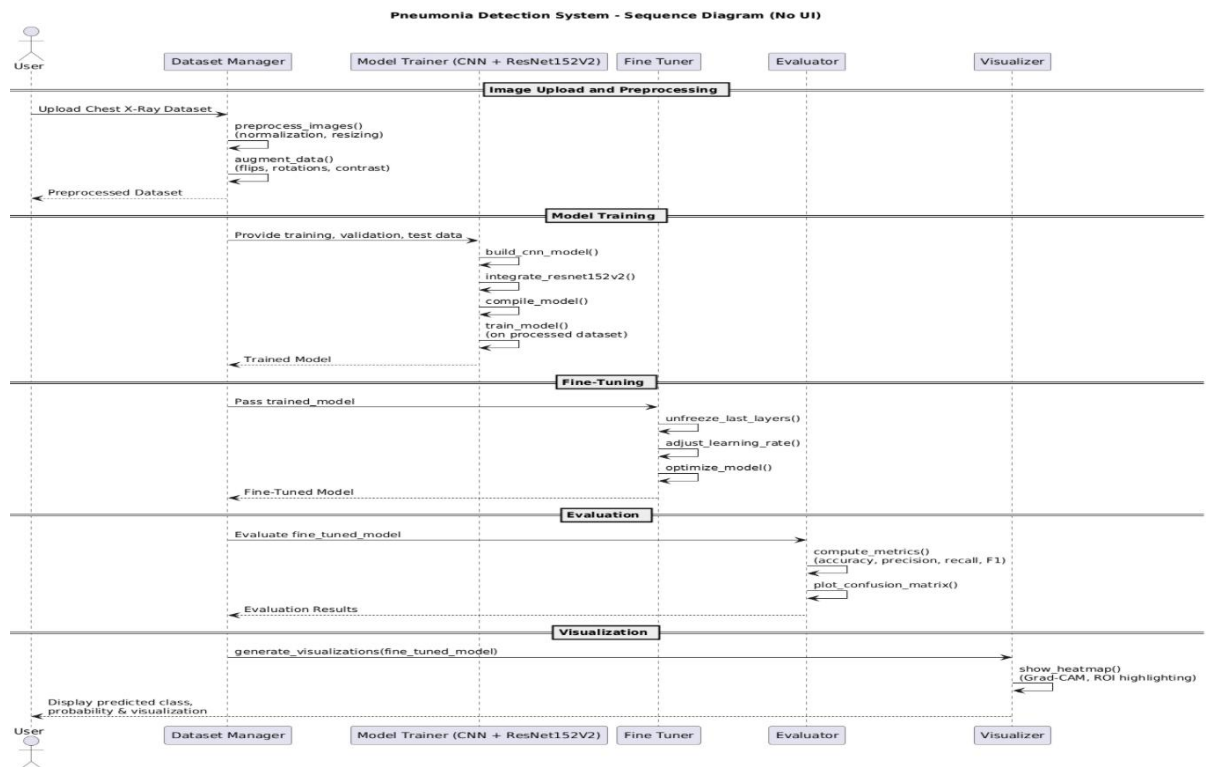


Fig. Sequence Diagram

## **CHAPTER 5**

### **METHODOLOGY AND TESTING**

#### **5.1 Module Description**

The pneumonia detection system is divided into several functional modules, each handling a specific stage of the workflow:

##### **1. Data Input Module**

- Responsible for accepting chest X-ray images from the user.
- Validates images for format, resolution, and integrity.

##### **2. Preprocessing Module**

- Resizes images to a fixed size (224x224) for model compatibility.
- Normalizes pixel values to [0,1] and applies data augmentation (rotation, flip, zoom).

##### **3. Segmentation Module (Optional)**

- Uses a U-Net-based network to extract lung regions.
- Applies morphological operations to refine masks and reduce noise.

##### **4. Feature Extraction Module**

- Custom CNN extracts hierarchical features from X-ray images.
- ResNet50V2 with transfer learning captures deep representations.

##### **5. Classification Module**

- Fully connected layers map extracted features to two classes: Normal or Pneumonia.
- Generates prediction probabilities and labels for each input image.

##### **6. Post-processing Module**

- Produces Grad-CAM heatmaps to visualize areas influencing predictions.
- Generates reports with predicted class, probability score, and annotated images.

#### **5.2 Testing**

Testing is performed to validate the system's reliability, accuracy, and robustness:

##### **1. Unit Testing**

- Each module (preprocessing, segmentation, CNN, ResNet) is tested individually.

- Ensures correct resizing, normalization, segmentation mask generation, and feature extraction.
2. **Integration Testing**
    - All modules are connected to simulate the full pipeline.
    - Confirms that data flows correctly from input to output without errors.
  3. **Model Evaluation**
    - **Metrics Used:** Accuracy, Precision, Recall, F1-Score, ROC-AUC.
    - Performance is measured on separate training, validation, and test datasets.
    - Confusion matrix is analyzed to identify common misclassifications.
  4. **Stress Testing**
    - System is tested with large batches of X-ray images to evaluate processing time.
    - Ensures scalability for real-time or bulk predictions.
  5. **Result Interpretation Testing**
    - Grad-CAM visualizations are checked for clinical relevance.
    - Ensures the model is not relying on irrelevant regions for prediction.

## **CHAPTER 6**

### **PROJECT DEMONSTRATION**

The system initially receives chest X-ray images. The preprocessing of these images involves resizing them to a standard input size suitable for CNNs, normalizing pixel intensity values to a value between 0 and 1 to improve training stability, and optionally segmenting the lung regions. The images thus preprocessed are consistent with each other and suitable for feature extraction by the models.

On completion of preprocessing, the pictures are submitted to two custom CNN models each uniquely different in terms of convolutional, pooling, and fully connected layer configurations. These custom models are trained from scratch on the dataset to extract hierarchical features from the X-ray images. Each CNN's architecture is optimized to identify patterns associated with pneumonia while maintaining generalization to unseen images.

Besides the custom CNNs, the system is using ResNet152V2 along with fine-tuning. The ResNet152V2 model, pretrained on ImageNet, is fine-tuned using the pneumonia dataset. Fine-tuning lets the model keep the general image features while it learns the pneumonia-specific ones, thus raising the accuracy and cutting down the overfitting. The hybrid strategy of comparing the two custom CNNs with the pretrained ResNet152V2 makes sure that the system evaluator looks into not only the traditional but also the state-of-the-art architectures for performance.

Throughout the presentation of the models, graphs are drawn to depict how well the models performed. These graphs include:

**Training and Validation Accuracy Curves:** Indicate model improvements over epochs and whether they effectively converge.

**Training and Validation Loss Curves:** Indicate learning efficiency and give hints to overfitting or underfitting issues.

**Confusion Matrices:** Show the real positive cases, real negative cases, false positive cases,

and false negative cases, thus giving the possibility of a detailed evaluation of the model classification performance.

The system demonstrates robustness and scalability by allowing batch mode evaluation of several images. It is possible to compare the performance of the two custom CNNs and the fine-tuned ResNet152V2 by looking at the plotted graphs. Such visualization helps in determining which model behaves best on the given dataset and also reveals each model's strengths and weaknesses.

The demonstration realizes the system potential to correctly categorize X-ray images into Normal or Pneumonia, deliver quantitative performance metrics, and generate abundant visuals that facilitate the comprehension of model behavior. Besides, it is a validation of the integrated pipeline from preprocessing to classification and performance evaluation, which is the main gist of the effectiveness of combining multiple deep learning models and fine-tuning strategies.

## CHAPTER 7

### RESULTS AND DISCUSSIONS

The pneumonia detection system was evaluated using three deep learning architectures: Custom CNN 1, Custom CNN 2 (Residual + SE blocks), and ResNet152V2 (pretrained with optional fine-tuning). Evaluation metrics include accuracy and loss on validation and test datasets.

| Model                    | Validation Accuracy | Validation Loss | Test Accuracy | Test Loss |
|--------------------------|---------------------|-----------------|---------------|-----------|
| Custom CNN 1             | 86.85%              | 0.4599          | 79.58%        | 0.4599    |
| Custom CNN 2 (Res+SE)    | 95.29%              | 0.1191          | 89.54%        | 0.2893    |
| ResNet152V2 (Pretrained) | 93.76%              | 0.1679          | 90.19%        | 0.2716    |
| ResNet152V2 (Fine-Tuned) | 96.06%              | 0.1182          | 92.28%        | 0.1999    |

#### Custom CNN 1

It achieved a test accuracy of 79.58% with moderate performance.

Validation accuracy of 86.85% shows some overfitting.

Limited depth and feature extraction capability make it less robust on unseen data.

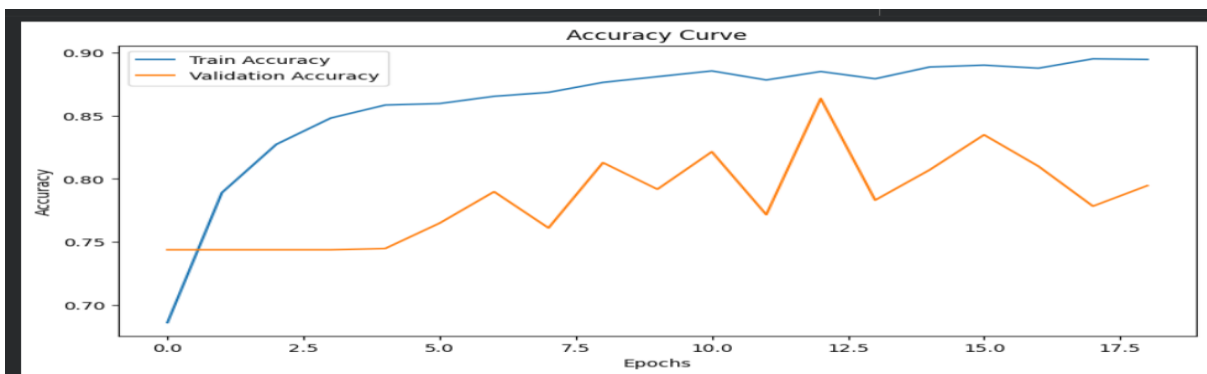


Fig. Accuracy Curve Of Custom CNN1

```
# Evaluate on Test Data
score = model.evaluate(test_gen)
print(f"\nTest Loss: {score[0]:.4f}")
print(f"Test Accuracy: {score[1]*100:.2f}%")

... 20/20 ----- 308s 16s/step - accuracy: 0.8719 - loss: 0.4976

Test Loss: 0.6528
Test Accuracy: 79.58%

score = model.evaluate(val_gen)
print(f"\nVal Loss: {score[0]:.4f}")
print(f"Val Accuracy: {score[1]*100:.2f}%")

<>:2: SyntaxWarning: invalid escape sequence '\V'
<>:2: SyntaxWarning: invalid escape sequence '\V'
/tmp/ipython-input-2891445547.py:2: SyntaxWarning: invalid escape sequence '\V'
print(f"\nVal Loss: {score[0]:.4f}")
33/33 ----- 27s 811ms/step - accuracy: 0.8727 - loss: 0.4896
Val Loss: 0.4599
Val Accuracy: 86.85%
```

Fig. Accuracy and Loss Of Custom CNN1

### Custom CNN 2 (Residual + SE Blocks)

Residual connections and SE blocks significantly improve performance.

Test accuracy reached 89.54%, while validation accuracy rose to 95.29%.

SE blocks focus on relevant features, improving generalization and reducing overfitting.

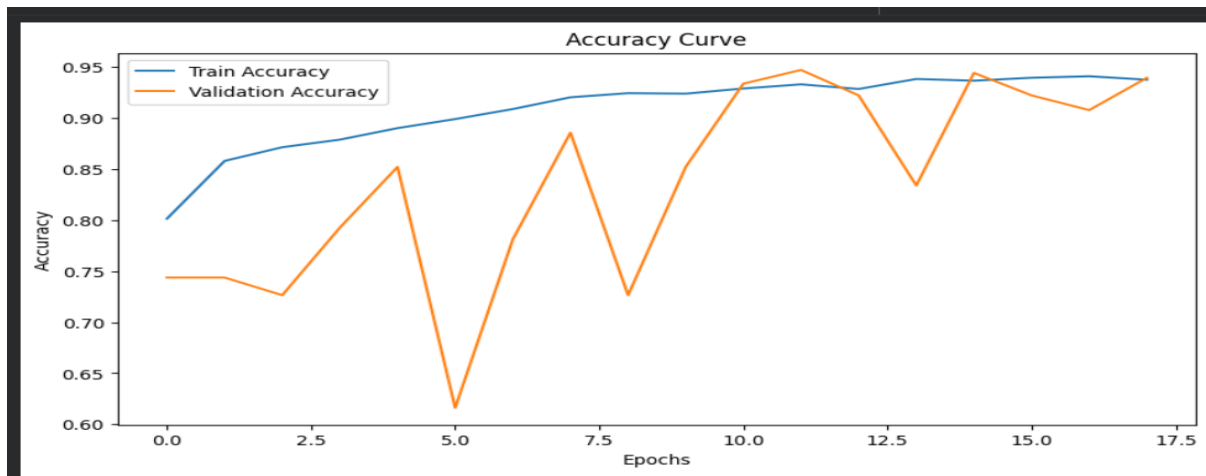


Fig. Accuracy Curve Of Custom CNN2

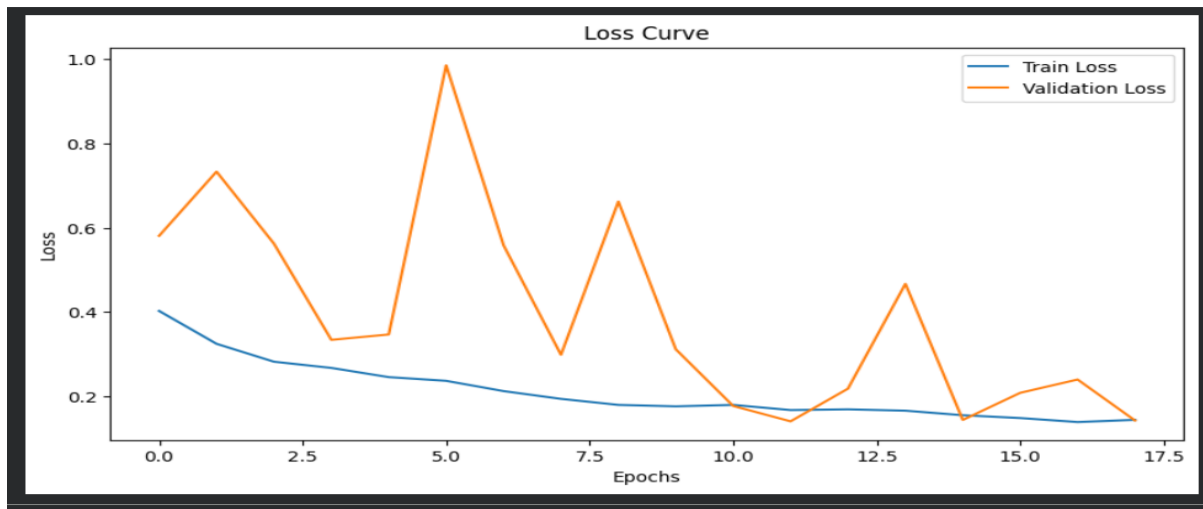


Fig. Loss Curve Of Custom CNN2

```
score = model.evaluate(val_gen, verbose = 1)
print('Val loss:', score[0])
print('Val accuracy:', score[1])
```

**33/33** ————— **27s** 820ms/step - accuracy: 0.9439 - loss: 0.1377  
Val loss: 0.11917208135128021  
Val accuracy: 0.9529750347137451

```
score = model.evaluate(test_gen, verbose = 1)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
```

**20/20** ————— **12s** 576ms/step - accuracy: 0.8889 - loss: 0.3053  
Test loss: 0.28935810923576355  
Test accuracy: 0.8954983949661255

Fig. Accuracy and Loss Score On CNN2

### ResNet152V2 (Pretrained)

Pre-trained weights provide rich feature extraction from ImageNet.

Test accuracy of 90.19% was achieved, slightly higher compared with Custom CNN 2.

This demonstrates that pre-trained models may perform well with no further fine-tuning.

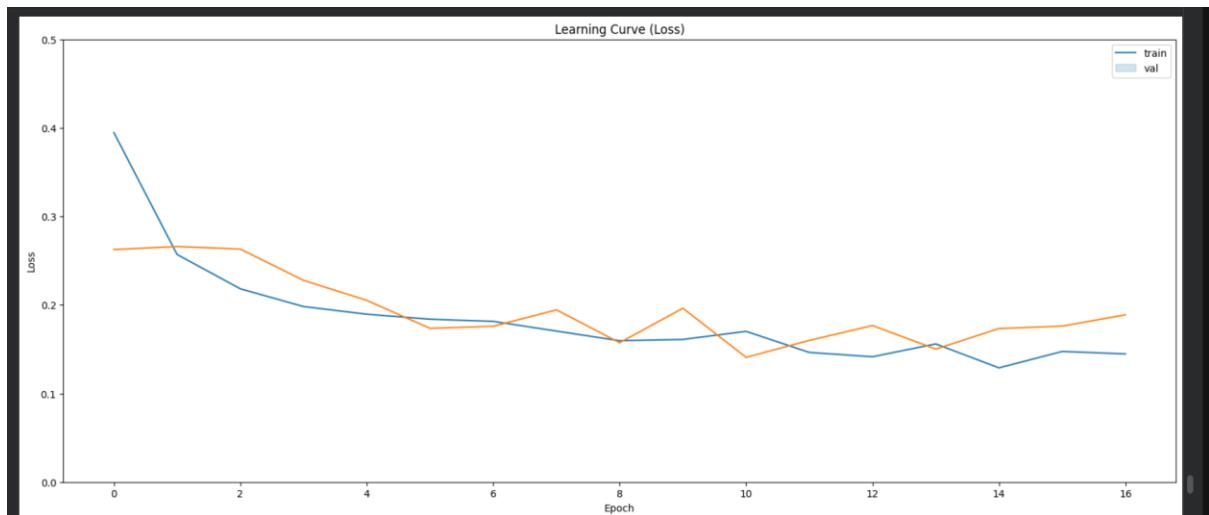


Fig. Loss Curve Of ResNet152v2(Pretrained)

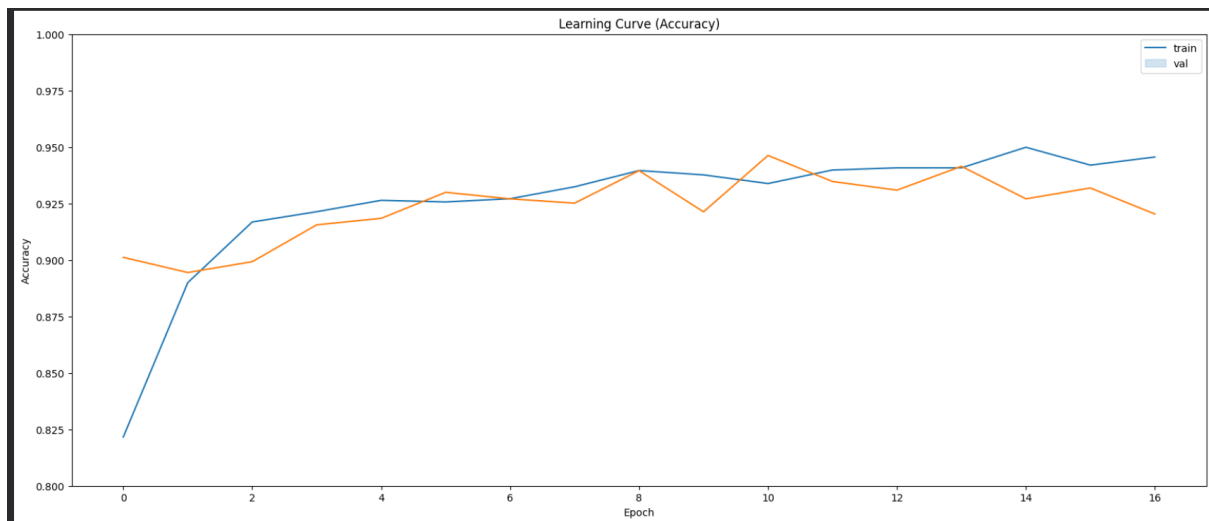


Fig. Accuracy Curve Of ResNet152v2(Pretrained)

```

score = model.evaluate(val_gen, verbose=1)
print('Val loss:', score[0])
print('Val accuracy:', score[1])

33/33 ————— 28s 846ms/step - accuracy: 0.9283 - loss: 0.1882
Val loss: 0.16796444356441498
Val accuracy: 0.9376199841499329

score = model.evaluate(test_gen, verbose = 1)
print('Test loss:', score[0])
print('Test accuracy:', score[1])

20/20 ————— 14s 706ms/step - accuracy: 0.9005 - loss: 0.2865
Test loss: 0.2716237008571625
Test accuracy: 0.9019292593002319

```

Fig. Accuracy and Loss Score Of ResNet152v2(Pretrained)

## Fine Tuned ResNet

Fine-tuning was carried out by unfreezing deeper layers of ResNet152V2 and retraining with a lower learning rate to optimize pneumonia-specific features. This further improved the accuracy while reducing validation loss.

Fine-tuning enhanced the model's ability to generalize and accurately detect pneumonia cases. The improvement in both validation and test accuracy reflects the effectiveness of layer-wise retraining.

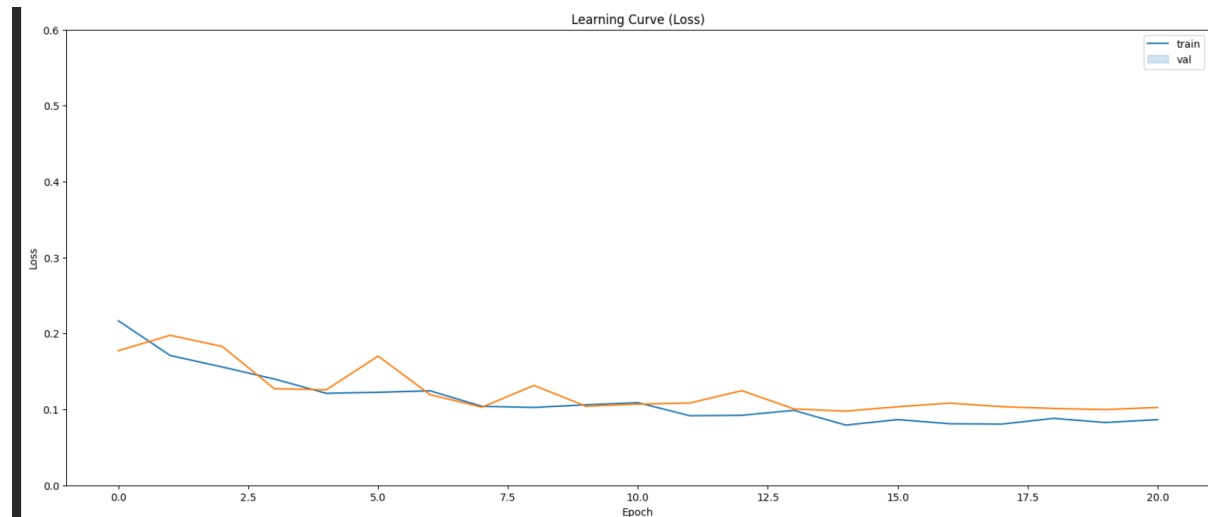


Fig. Loss Curve of Fine Tuned ResNet

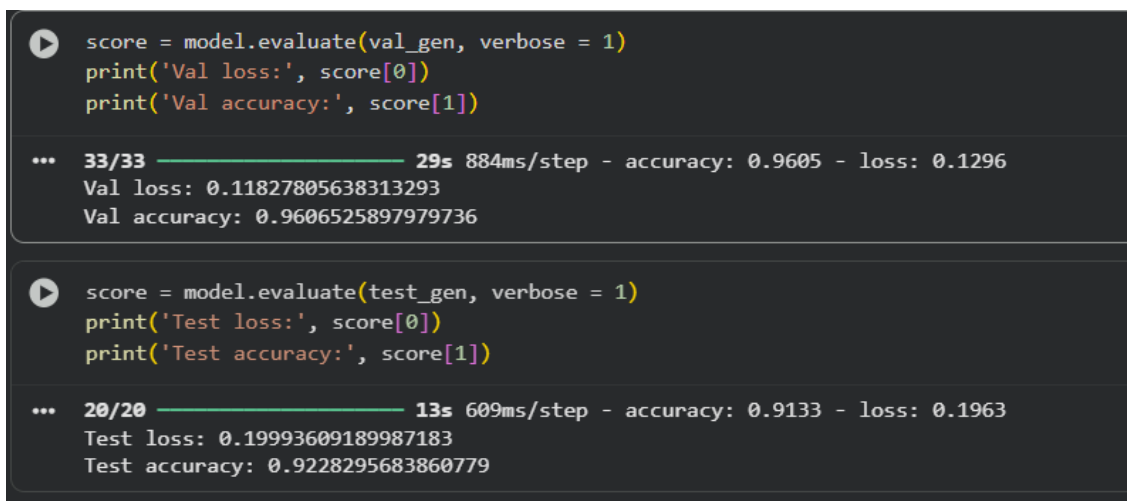


Fig. Accuracy and Loss Score of Fine Tuned ResNet

## **CHAPTER 8**

### **CONCLUSIONS**

The pneumonia detection system focuses on the efficiency of deep learning-based architectures in developing either a Normal or Pneumonia classification for chest X-ray images. Comparison of the results has been made for three systems: Custom CNN 1, Custom CNN 2 with Residual and SE blocks, and ResNet152V2, pre-trained (with optional fine tuning).

Key conclusions from the project include:

Custom CNN Performance:

Custom CNN 1 had a moderate performance, which may suggest that the simple architecture of CNN is not enough for complex medical image classification.

Custom CNN 2, enhanced by incorporating residual connections and SE blocks, demonstrated significant improvements in feature extraction for better accuracy and generalization.

Pretrained Models:

ResNet152V2 pre-trained on ImageNet showed remarkable performance without fine-tuning, demonstrating the power of transfer learning in medical image classification.

Fine-tuning ResNet152V2 will most probably further improve performance by adapting the pre-trained features to pneumonia-specific patterns.

Model Comparison:

Of all the tested models, Custom CNN 2 and ResNet152V2 outperformed Custom CNN 1 on both the validation and test datasets.

Residual connections, SE blocks, and pre-trained feature extraction turned out to be very

responsible for such classification accuracy with reduced overfitting.

Overall Impact:

This system provides an automatic, reliable, scalable pneumonia detection solution that will help healthcare professionals with early diagnosis.

Accuracy and loss curves represent performance visualization, by which model stability and convergence can be judged to assure result interpretability.

## CHAPTER 9

### REFERENCES

1. R. Kailasam et al., "Deep Learning for Pneumonia Detection: A Combined Approach," *Journal of Medical Imaging*, 2025, SpringerLink.
2. A. B. Rashid et al., "An Enhanced Deep Learning Framework for Pneumonia Detection in Chest X-rays," *SpringerLink*, 2025.
3. F. Alshanketi, "Pneumonia Detection from Chest X-Ray Images Using Deep Learning," *Journal of Digital Imaging*, 2024, SpringerLink.
4. L. Gowri, "Enhancing Pneumonia Detection with Masked Neural Networks," *Neural Computing and Applications*, 2024, SpringerLink.
5. S. Sharma, "A Systematic Literature Review on Deep Learning Techniques for Pneumonia Detection," *Journal of Ambient Intelligence and Humanized Computing*, 2023, SpringerLink.
6. A. Mehta and M. Vyas, "Deep Learning for Pneumonia Diagnosis: A Custom CNN Approach with Superior Performance on Chest Radiographs," *Radiology AI Research Hub*, 2025.
7. P. R. Dave, Y. B. Patel and S. R. Patel, "Comparative Analysis of Customized CNN and Teachable Machine for Pneumonia Detection in Chest X-Ray Images," *ShodhKosh – Journal of Visual & Performing Arts*, vol. 5, no. 3, 2024.
8. N. M. E. and D. M. I., "Deep-Pneumonia Framework Using Deep Learning Models Based on Chest X-Ray Images," *Diagnostics*, vol. 10, no. 9, p. 649, 2020.
9. C. Wang, "A Literature Review of Pneumonia Detection Algorithms Based on Deep Learning and Chest X-Ray Imaging," *Science and Technology of Engineering, Chemistry and Environmental Protection*, vol. 1, no. 1, 2024.
10. J. Imaging, "Deep Learning for Pneumonia Detection in Chest X-Ray Images: A Comprehensive Survey," *Journal of Imaging*, vol. 10, no. 8, p. 176, 2024.
11. Y. Yao, Y. Li, S. Zheng and T. Zhu, "Deep Learning Models for Multi-Class Pneumonia Detection in Chest X-Rays: A Comparative Study of VGG16, MobileNet, and ResNet152," *Preprints.org*, 2025.
12. J. Roman Sarkar, S. M. Sojib Ahamed, M. Miftahul Bari and M. Mehedi Hasan, "Deep Learning Methods for Chest X-Ray Imaging-Based COVID-19 Pneumonia Detection," *Journal of Image Processing and Intelligent Remote Sensing*, 2025.

13. S. Kora Venu, “An Ensemble-Based Approach by Fine-Tuning the Deep Transfer Learning Models to Classify Pneumonia from Chest X-Ray Images,” *arXiv preprint*, arXiv:2011.05543, 2020.
14. S. N. Roy, A. Suresh, P. Sahu and T. Rudra Gupta, “Novel Pooling-based VGG-Lite for Pneumonia and COVID-19 Detection from Imbalanced Chest X-Ray Datasets,” *arXiv preprint*, arXiv:2504.07468, 2025.
15. “Efficient Pneumonia Detection in Chest X-Ray Images Using Deep Transfer Learning,” *Healthcare*, 2024.

## APPENDIX A- Sample Code

```
import os, glob
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
IMG_SIZE = 224
BATCH = 64
SEED = 42
from google.colab import drive
drive.mount('/content/drive')

import os, glob

main_path = "/content/drive/MyDrive/pneumonia/final project/chest_xray"

train_path = os.path.join(main_path, "train")
test_path = os.path.join(main_path, "test")
main_path = "/content/drive/MyDrive/pneumonia/final project/chest_xray"

train_path = os.path.join(main_path, "train")
test_path = os.path.join(main_path, "test")

train_normal = glob.glob(train_path + "/NORMAL/*")
train_pneumonia = glob.glob(train_path + "/PNEUMONIA/*")
test_normal = glob.glob(test_path + "/NORMAL/*")
test_pneumonia = glob.glob(test_path + "/PNEUMONIA/*")

print("Train NORMAL:", len(train_normal))
print("Train PNEUMONIA:", len(train_pneumonia))
print("Test NORMAL:", len(test_normal))
print("Test PNEUMONIA:", len(test_pneumonia))

import pandas as pd
import numpy as np
train_list = [x for x in train_normal]
train_list.extend([x for x in train_pneumonia])

df_train = pd.DataFrame(np.concatenate([['Normal']*len(train_normal), ['Pneumonia']*len(train_pneumonia)]), columns = ['class'])
df_train['image'] = [x for x in train_list]
```

```

test_list = [x for x in test_normal]
test_list.extend([x for x in test_pneumonia])

df_test = pd.DataFrame(np.concatenate([['Normal']*len(test_normal) , ['Pneumonia']*len(test_pneumonia)]), columns = ['class'])
df_test['image'] = [x for x in test_list]
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))

ax = sns.countplot(x='class', data=df_train, palette="mako")

plt.xlabel("Class", fontsize= 12)
plt.ylabel("# of Samples", fontsize= 12)
plt.ylim(0,5000)
plt.xticks([0,1], ['Normal', 'Pneumonia'], fontsize = 11)

for p in ax.patches:
    ax.annotate((p.get_height()), (p.get_x()+0.30, p.get_height()+300), fontsize = 13)

plt.show()
plt.figure(figsize=(7,5))

df_train['class'].value_counts().plot(kind='pie', labels = ["", ""], autopct='%1.1f%%', colors = ['darkcyan','blue'], explode = [0,0.05], textprops =
{"fontsize":15})

plt.legend(labels=['Pneumonia', 'Normal'])
plt.show()

plt.figure(figsize=(6,4))

ax = sns.countplot(x='class', data=df_test, palette="mako")

plt.xlabel("Class", fontsize= 12)
plt.ylabel("# of Samples", fontsize= 12)
plt.ylim(0,500)
plt.xticks([0,1], ['Normal', 'Pneumonia'], fontsize = 11)

for p in ax.patches:
    ax.annotate((p.get_height()), (p.get_x()+0.32, p.get_height()+20), fontsize = 13)

plt.show()
import cv2
print('Train Set - Normal')

plt.figure(figsize=(12,12))

```

```

for i in range(0, 12):
    plt.subplot(3,4,i + 1)
    img = cv2.imread(train_normal[i])
    img = cv2.resize(img, (IMG_SIZE,IMG_SIZE))
    plt.imshow(img)
    plt.axis("off")

plt.tight_layout()

plt.show()
print('Train Set - Pneumonia')

plt.figure(figsize=(12,12))

for i in range(0, 12):
    plt.subplot(3,4,i + 1)
    img = cv2.imread(train_pneumonia[i])
    img = cv2.resize(img, (IMG_SIZE,IMG_SIZE))
    plt.imshow(img)
    plt.axis("off")

plt.tight_layout()

plt.show()

# Data Generators
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=25,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.3,
    horizontal_flip=True,
    brightness_range=[0.8, 1.2],
    validation_split=0.2
)

test_datagen = ImageDataGenerator(rescale=1./255)

train_gen = train_datagen.flow_from_directory(
    train_path,
    target_size=(224,224),
    batch_size=32,
    class_mode='binary',

```

```

        subset='training'
    )

val_gen = train_datagen.flow_from_directory(
    train_path,
    target_size=(224,224),
    batch_size=32,
    class_mode='binary',
    subset='validation'
)

test_gen = test_datagen.flow_from_directory(
    test_path,
    target_size=(224,224),
    batch_size=32,
    class_mode='binary',
    shuffle=False
)

# CNN Model
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Dropout(0.25),

    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Dropout(0.3),

    Conv2D(128, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Dropout(0.4),

    Conv2D(256, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Dropout(0.4),

    Flatten(),
    Dense(512, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

```

```

model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-4),
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True)
lr_reduce = ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=3, min_lr=1e-6)
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.array([0,1]),
    y=train_gen.classes
)

class_weights = {0: class_weights[0], 1: class_weights[1]}
print(class_weights)

history = model.fit(
    train_gen,
    epochs=40,
    validation_data=val_gen,
    callbacks=[early_stop, lr_reduce],
    class_weight=class_weights
)

# Evaluate on Test Data
score = model.evaluate(test_gen)
print(f"\nTest Loss: {score[0]:.4f}")
print(f"Test Accuracy: {score[1]*100:.2f}%")

score = model.evaluate(val_gen)
print(f"\nVal Loss: {score[0]:.4f}")
print(f"Val Accuracy: {score[1]*100:.2f}%")

plt.figure(figsize=(10,5))
sns.lineplot(x=range(len(history.history['accuracy'])),
             y=history.history['accuracy'], label='Train Accuracy')
sns.lineplot(x=range(len(history.history['val_accuracy'])),
             y=history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Curve')
plt.xlabel('Epochs')

```

```

plt.ylabel('Accuracy')
plt.legend()
plt.show()
import tensorflow as tf
from tensorflow.keras.layers import *
from tensorflow.keras.models import Model

# ----- Squeeze-and-Excitation Block -----
def se_block(x, r=16):
    c = x.shape[-1]
    se = GlobalAveragePooling2D()(x)
    se = Dense(c // r, activation='relu')(se)
    se = Dense(c, activation='sigmoid')(se)
    se = Multiply()(x, se)
    return se

# ----- Residual Block + SE Block -----
def res_block(x, filters):
    shortcut = x

    x = Conv2D(filters, (3,3), padding='same', activation='relu')(x)
    x = BatchNormalization()(x)

    x = Conv2D(filters, (3,3), padding='same')(x)
    x = BatchNormalization()(x)

    if shortcut.shape[-1] != filters:
        shortcut = Conv2D(filters, 1, padding='same')(shortcut)
    x = se_block(x)    # SE block

    x = Add()(x, shortcut) # Residual connection
    x = Activation('relu')(x)
    return x

# ----- MAIN CNN -----
def build_model(input_shape=(224,224,3)):
    inputs = Input(input_shape)

    # Stem
    x = Conv2D(32, 3, padding='same', activation='relu')(inputs)
    x = BatchNormalization()(x)

    # Stage 1
    x = res_block(x, 32)
    x = MaxPooling2D()(x)

```

```

# Stage 2
x = res_block(x, 64)
x = MaxPooling2D()(x)

# Stage 3
x = res_block(x, 128)
x = MaxPooling2D()(x)

# Stage 4
x = res_block(x, 256)
x = MaxPooling2D()(x)

# Head
x = GlobalAveragePooling2D()(x)
x = Dropout(0.4)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.3)(x)
outputs = Dense(1, activation='sigmoid')(x)

model = Model(inputs, outputs)
return model

model = build_model()
model.summary()

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_gen,
    epochs=50,
    validation_data=val_gen,
    callbacks=[early_stop, lr_reduce],
    class_weight=class_weights
)

# PLOT RESULTS
# =====
plt.figure(figsize=(10,5))
sns.lineplot(x=range(len(history.history['accuracy'])),
             y=history.history['accuracy'], label='Train Accuracy')
sns.lineplot(x=range(len(history.history['val_accuracy'])),
             y=history.history['val_accuracy'], label='Validation Accuracy')

```

```

plt.title('Accuracy Curve')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
plt.figure(figsize=(10,5))
sns.lineplot(x=range(len(history.history['loss'])),
             y=history.history['loss'], label='Train Loss')
sns.lineplot(x=range(len(history.history['val_loss'])),
             y=history.history['val_loss'], label='Validation Loss')
plt.title('Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

```

```

base_model = tf.keras.applications.ResNet152V2(
    weights='imagenet',
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False)

```

```
base_model.trainable = False
```

```
def get_pretrained():
```

```

    inputs = keras.layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
    x = base_model(inputs)

```

```

    x = keras.layers.GlobalAveragePooling2D()(x)
    x = keras.layers.Dense(128, activation='relu')(x)
    x = keras.layers.Dropout(0.1)(x)

```

```
output = keras.layers.Dense(1, activation='sigmoid')(x)
```

```

model = keras.Model(inputs, output)
return model

```

```
model = get_pretrained()
```

```

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

```

```

history = model.fit(
    train_gen,

```

```

    epochs=50,
    validation_data=val_gen,
    callbacks=[early_stop, lr_reduce],
    class_weight=class_weights
)
fig, ax = plt.subplots(figsize=(20,8))
sns.lineplot(x = history.epoch, y = history.history['loss'])
sns.lineplot(x = history.epoch, y = history.history['val_loss'])
ax.set_title('Learning Curve (Loss)')
ax.set_ylabel('Loss')
ax.set_xlabel('Epoch')
ax.set_ylim(0, 0.5)
ax.legend(['train', 'val'], loc='best')
plt.show()
fig, ax = plt.subplots(figsize=(20,8))
sns.lineplot(x = history.epoch, y = history.history['accuracy'])
sns.lineplot(x = history.epoch, y = history.history['val_accuracy'])
ax.set_title('Learning Curve (Accuracy)')
ax.set_ylabel('Accuracy')
ax.set_xlabel('Epoch')
ax.set_ylim(0.80, 1.0)
ax.legend(['train', 'val'], loc='best')
plt.show()
base_model.trainable = False
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
history = model.fit(train_gen, validation_data=val_gen, epochs=5)

# Unfreeze last 40 layers
for layer in base_model.layers[:-40]:
    layer.trainable = False
for layer in base_model.layers[-40:]:
    layer.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_gen,
    validation_data=val_gen,

```

```

    epochs=30,
    callbacks=[early_stop, lr_reduce],
    class_weight=class_weights
)

fig, ax = plt.subplots(figsize=(20,8))
sns.lineplot(x = history.epoch, y = history.history['loss'])
sns.lineplot(x = history.epoch, y = history.history['val_loss'])
ax.set_title('Learning Curve (Loss)')
ax.set_ylabel('Loss')
ax.set_xlabel('Epoch')
ax.set_ylim(0, 0.6)
ax.legend(['train', 'val'], loc='best')
plt.show()

```